

Data Structures and Algorithm

COURSE CODE: CS-201

TOPIC: SINGLY LINKED DELETION



Goals for today

To study:

- Singly Linked List
 - Deletion

CLO Covered

CLO2: **Apply** and design various data structures methods for specified applications.

OPERATION ON SINGLE LINKED LIST

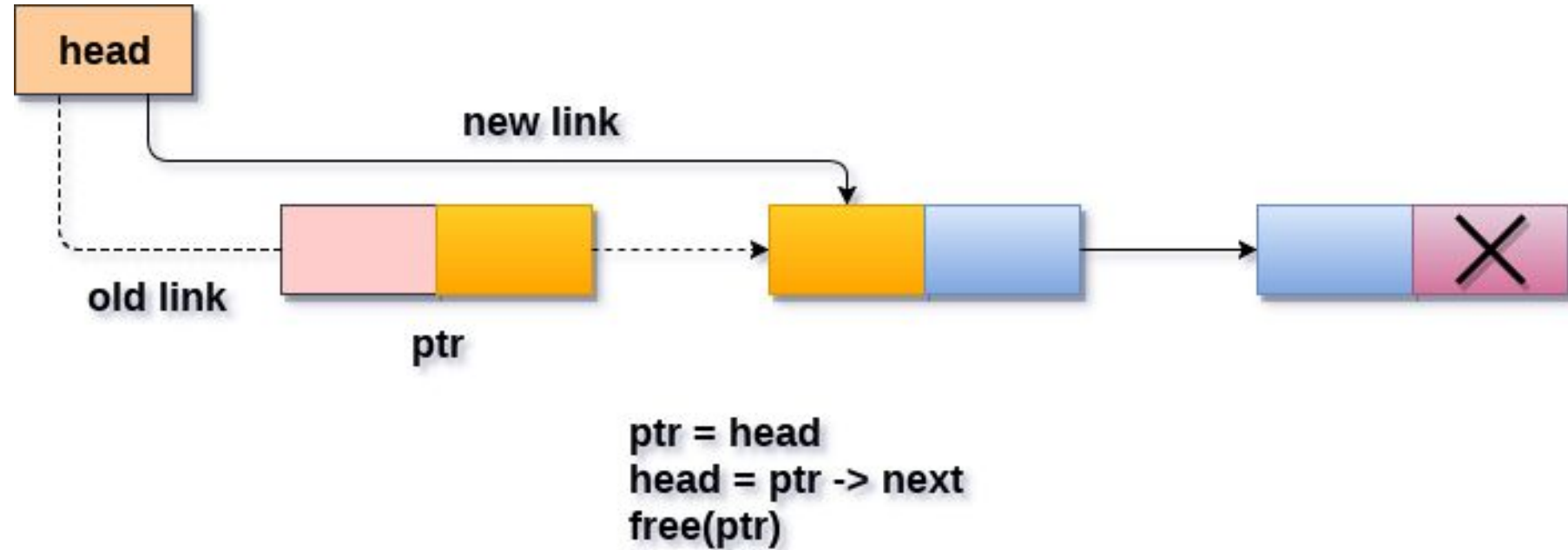
Deletion an element from the linked list

If the node be deleted, that element should be search all over the list still the node find. Then it should be deleted.

As like as insertion the deletion is also performed in three ways.

- Deletion at the beginning position of the linked list.
- Deletion at the ending position of linked list.
- Deletion at a specified position in linked list.

Deletion at beginning



Deletion at beginning

Step 1: If *Start* = *Null* then

 Print: “linked list is already empty-Underflow”

 Exit [End if]

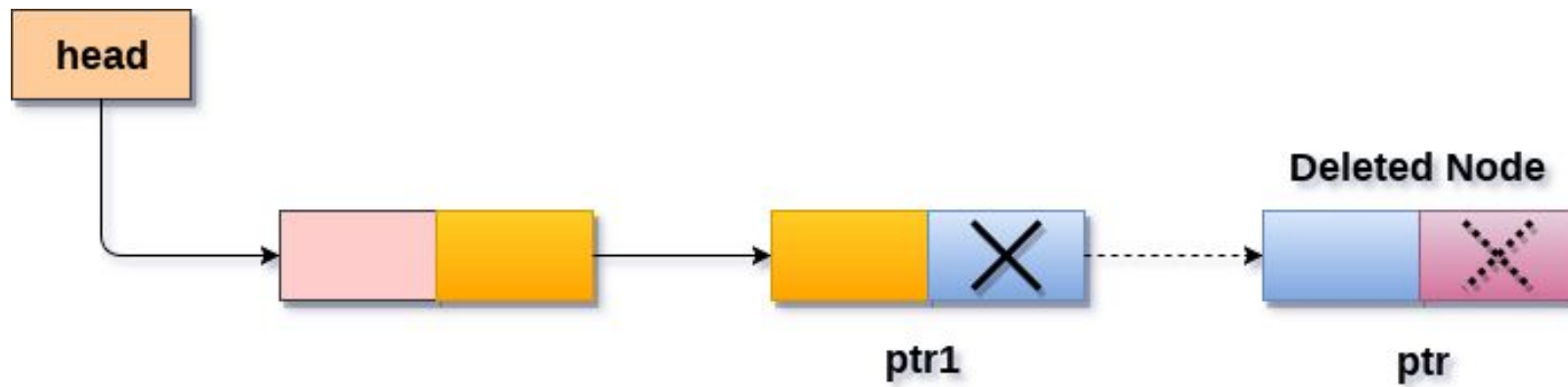
Step 2: set *Ptr* = *Start*

Step 3: set *Start* = *Start* → *Next*

Step 4: free(*Ptr*)

Step 5: Exit

Deletion At end



Deletion At end

Step 1: If *Start* = *Null* Then

 Print : “Linked List Empty”

 Exit

 [End If]

Step 2: set *Ptr* = *Start*

Step 3: Repeat steps 4 and 5 while *Ptr* → *next* ≠ *Null*

 Step 4: set *temp* = *Ptr*

 Step 5: set *Ptr* = *Ptr* → *next*

 [End loop]

Step 6: Set *temp* → *next* = *Null*

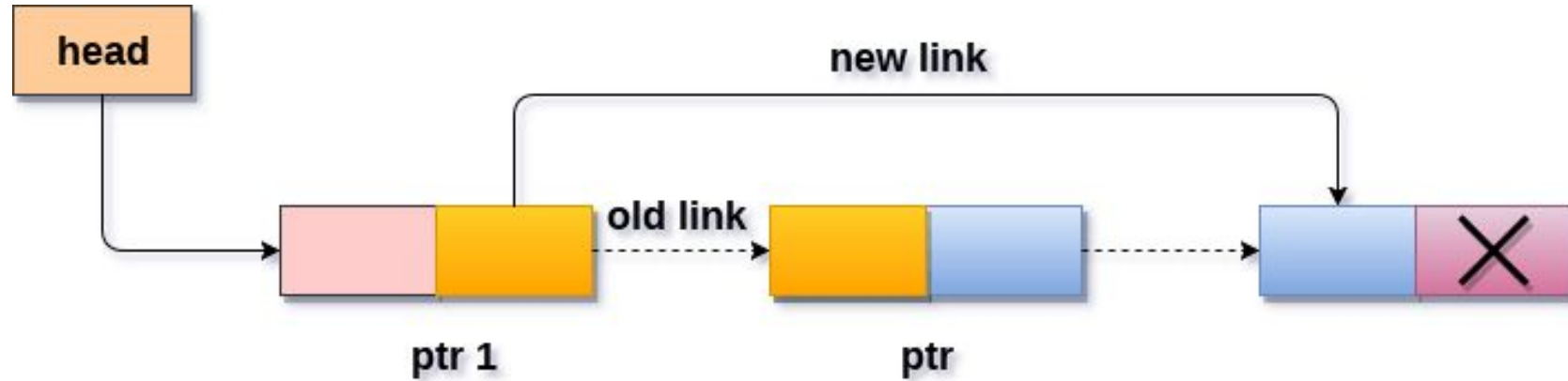
Step 7: *free*(*Ptr*)

Step 8: Exit

Deletion at the specified position

- For deleting a particular node from the linked list, the first task is to find the address of the preceding node of nth node to be deleted.
- To complete the task traverse the linked list from begin and compare the info. Stored in node with item.
- Two pointers pointer1 pointer2 will be used while traversing the list for locating the address of the node to be deleted and address of it's preceding node.

Deletion at the specified position



`ptr1 -> next = ptr -> next`
`free(ptr)`

Deletion at the specified position

```
struct node *p;  
struct node *tmp;  
p = start;  
while(p → link != Null)  
{  
    If(p → link → info == data)  
    {  
        tmp = p → link;  
        p → link = tmp → link;  
        free(tmp);  
    }  
    p = p → link;  
}
```

Questions?